

FACADE

JULIAN FELIPE CARDENAS ZIPASUCA

PATRONES DE DISEÑO DE SOFTWARE

UNINPAHU

2025

## **1. ¿Cuál es el patrón de diseño Facade?**

El patrón de diseño Facade es un patrón estructural que tiene como objetivo proporcionar una interfaz de más alto nivel y más simple para acceder a un conjunto de interfaces de un subsistema complejo. Este patrón actúa como una "fachada", es decir, una capa intermedia que encapsula la lógica y operaciones complicadas que se encuentran detrás del sistema, permitiendo que los clientes interactúen con él de forma más sencilla, sin necesidad de entender o manipular directamente todos sus componentes internos.

## **2. ¿Cuáles es el propósito principal del patrón Facade?**

El propósito principal del patrón Facade es ocultar la complejidad interna de un sistema y ofrecer una interfaz simplificada que facilite su uso. Al abstraer la complejidad del subsistema, los desarrolladores pueden interactuar con una única clase o punto de entrada en lugar de tener que lidiar con múltiples clases interconectadas. Además, este patrón contribuye a mantener el principio de separación de responsabilidades, ya que permite a los clientes centrarse en lo que necesitan hacer sin preocuparse por cómo se hace internamente.

## **3. ¿En qué tipo de situaciones se recomienda usar el patrón Facade?**

- Cuando un sistema es muy complejo o tiene muchas dependencias internas y se desea simplificar su uso para los clientes.
- Cuando se trabaja con un subsistema legado difícil de modificar, pero se quiere encapsular su funcionalidad con una nueva interfaz.

- Cuando se desea establecer una capa intermedia entre el cliente y los componentes del sistema para mejorar la modularidad y el desacoplamiento.
  
- En sistemas donde se requiere organizar mejor el código y separar claramente las responsabilidades.

#### **4. Menciona una ventaja de utilizar el patrón Facade**

Una de las principales ventajas del patrón Facade es que reduce el acoplamiento entre los clientes y el sistema interno. Esto significa que, si en el futuro se desea modificar, extender o incluso reemplazar partes del subsistema, no será necesario cambiar el código de los clientes, ya que ellos sólo interactúan con la fachada. Además, mejora la legibilidad del código, facilita las tareas de pruebas, documentación y mantenimiento, y permite establecer una interfaz pública más coherente y controlada.

#### **5. ¿Cómo se relaciona el patrón Facade con el principio de menor conocimiento?**

El principio de menor conocimiento establece que un objeto debe tener el menor conocimiento posible sobre otros objetos. El patrón Facade promueve este principio al actuar como intermediario entre los clientes y el subsistema. El cliente no necesita conocer todos los detalles internos ni cómo están estructurados los componentes del sistema. Solo necesita conocer la fachada, que centraliza la lógica de interacción. De esta forma, se minimiza la dependencia y la posibilidad de que los cambios en el sistema afecten al cliente.

## **6. ¿Qué elementos componen el patrón Facade?**

Clase Facade: Es la interfaz de alto nivel que proporciona métodos simples y comprensibles para interactuar con el sistema. Esta clase encapsula la complejidad de los componentes internos.

Subsistemas o Componentes Internos: Son las clases y módulos que contienen la lógica real del sistema. La fachada utiliza estos componentes para realizar su trabajo, pero los mantiene ocultos al cliente.

Cliente: Es el componente o usuario del sistema que necesita acceder a la funcionalidad del subsistema, pero lo hace a través de la fachada en lugar de interactuar directamente con las clases del subsistema.

## **7. Explica como implementar el patrón Facade en un sistema complejo**

Analizar el sistema para identificar las clases que conforman el subsistema complejo.

Diseñar una clase fachada que actúe como punto de acceso principal. Esta clase debe contener métodos que ofrezcan funcionalidades específicas al cliente.

Implementar en la fachada la lógica que coordina las interacciones entre los distintos componentes internos. La fachada no debe implementar la lógica del negocio, sino orquestar a los objetos del subsistema.

Reemplazar las llamadas directas al subsistema por llamadas a la fachada. A partir de ese momento, los clientes solo interactúan con la fachada.

## **8. ¿Qué diferencias existen entre Facade y un Adapter?**

El patrón Facade proporciona una interfaz nueva y simplificada para un sistema completo o subsistema. Se enfoca en simplificar la interacción con múltiples clases complejas.

El patrón Adapter se usa para hacer que una clase con una interfaz existente incompatible pueda ser usada por otra clase o sistema que espera una interfaz diferente. En otras palabras, traduce una interfaz a otra.

## **9. ¿Cómo contribuye el patrón Facade a la mantenibilidad del código?**

Aísla los cambios internos del sistema. Si un componente interno cambia, solo la fachada debe actualizarse, no todos los clientes.

Reduce el acoplamiento entre componentes, lo que facilita modificar, reemplazar o actualizar módulos sin afectar a otros, organiza mejor la arquitectura del software, permitiendo una separación más clara entre la lógica de negocio y la lógica de presentación o interfaz de usuario, facilita las pruebas y depuración, ya que se puede probar la fachada de forma aislada, sin necesidad de conocer todos los detalles del subsistema.

## **10. Da un ejemplo práctico donde usarías el patrón Facade**

Un buen ejemplo es un sistema de automatización del hogar, como una casa inteligente. Supongamos que el usuario quiere activar el modo “fuera de casa”, lo que implica apagar las luces, cerrar las puertas, activar las cámaras de seguridad, bajar el termostato, entre otras cosas. Sin una fachada, el usuario o desarrollador tendría que llamar a cada clase por separado, como `luces.apagar()`, `cerraduras.cerrar()`, `camaras.activar()`, `termostato.bajarTemperatura()`. Esto no solo es engorroso, sino difícil de mantener. Con el patrón Facade, se puede crear una clase llamada `SmartHomeFacade` con un método llamado `activarModoFueraDeCasa()`. Este método se encarga internamente de ejecutar todas esas tareas en orden, y el cliente solo tiene que hacer una llamada simple. Esto simplifica el uso del sistema, facilita su mantenimiento y permite una mayor escalabilidad en el futuro.